GRADUATION PROJECT PRESENTATION - MODULE

Cortex System Architecture & Database Design

A secure, scalable collaborative notebook server and entity model for high-density academic workflows.

Presenter: Member 1 (Team Leader)

Core Focus: Architecture & Database Design

Menoufia University | Computer Science & Engineering Division

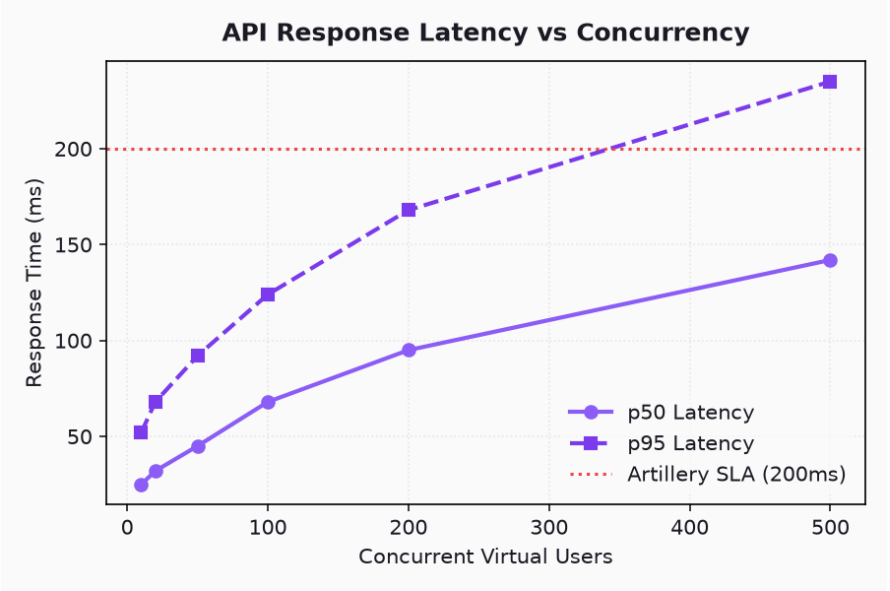
Project Scope & Productivity Challenges

- Students manage fragmented tools: rich text notes, calendar logs, resource uploads, and AI bots.
- Visual systems lack Arabic styling, while workspace groups lack role verification to prevent spam.
- Cortex unifies these needs: collaborative editor, structured resource registries, vector semantic search.
- Our platform includes browser and desktop apps to provide focus controls at the browser and OS layers.
- [SCREENSHOT: Dashboard showing scattered folders, browser focus blockers, and unfinished study sheets]

```
// Core Module Mappings
- backend/      (Express API Server)
- frontend/     (Next.js Web Client)
- extension/    (Chrome Focus Timer)
- gnome-ext/    (GNOME Shell Widget)
- supabase/     (DB Schemas & Policies)
```

Decoupled System Architecture Topology

- High-performance decoupled client-server model built for low latency and state syncing.
- Next.js App Router serves static layouts, requesting API routes asynchronously.
- Express TypeScript backend manages security filters, RAG queues, and SSE streaming channels.
- Supabase PostgreSQL handles user permissions, core storage, and pgvector embeddings.
- [SCREENSHOT: System deployment chart detailing client requests routed through Nginx to Node.js APIs]



Relational Schema & Catalogs

- Registry tables match university structure: Universities, Colleges, Departments, and year levels.
- Database entities maintain referential integrity with cascading deletes on child relations.
- User profiles link accounts to departments, styling themes, and verification queues.
- [SCREENSHOT: Database table visualizer showing profiles, majors, and university schema links]

Table Name	Primary Key	Relations	Indexed Columns
profiles	id (UUID)	auth.users	major_id, role
notes	id (UUID)	profiles.id	folder_id, is_pinned
note_chunks	id (UUID)	notes.id	embedding (hnsw)
resources	id (UUID)	courses.id	uploaded_by, type

User Security & Row-Level Security Rules

- Enforce security directly in the database using Supabase Row-Level Security policies.
- Profiles table blocks standard updates, checking the user's verified identity.

- Database schemas restrict note readability, unless shared via a secure note share entry.
- [SCREENSHOT: Database explorer panel showing active RLS policies enabled for public.notes]

```
-- Profiles DDL schema snippet
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id),
  name TEXT,
  role TEXT DEFAULT 'user',
  is_verified BOOLEAN DEFAULT FALSE,
  university_id UUID,
  major_id UUID
);
```

API Gateway & Express Routing Architecture

- The backend router exposes endpoints grouped by resource modules (auth, profile, notes, daily).
- Express routes are guarded by token validation middleware.
- Request rate limiters manage traffic to secure database query workloads.

- [SCREENSHOT: Terminal output of the Express server starting up, showing loaded routes]

```
// Express Gateway Router
import express from "express";
import { notesRouter } from "./routes/notes";
import { dailyRouter } from "./routes/daily";

const app = express();
app.use(express.json());

// Load secured router pipelines
app.use("/api/notes", notesRouter);
app.use("/api/daily", dailyRouter);
```

API Routing Guard Middleware

- AuthMiddleware decodes JWT keys using the Supabase key helper.
- Authorized claims are appended to incoming request contexts.
- Helmet security headers restrict unauthorized cross-origin requests.
- [SCREENSHOT: Postman requests showing 401 response status for missing JWT header]

```
// JWT Verification Middleware
export async function authMiddleware(
  req, res, next
) {
  const token = req.headers.authorization;
  if (!token) return res.status(401);

  try {
    const user = await supabase.auth.getUser(token);
    req.user = user;
    next();
  } catch (err) {
    return res.status(403);
  }
}
```

Express Server Middleware Pipeline

- Express integrates rate-limiting middleware to block denial-of-service attempts.
- Security settings block cross-origin requests, allowing browser extensions and web dashboards.
- Unified error-handling middleware catches database errors to prevent server crashes.
- [SCREENSHOT: Shell terminal showing rate-limit block responses during high concurrency load tests]

```
// Security Middlewares Config
import helmet from "helmet";
import rateLimit from "express-rate-limit";

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 mins
  max: 2000, // max requests
  message: { error: "Rate limit exceeded" }
});

app.use(helmet());
app.use(limiter);
```

Database pgvector Vector Index Tuning

- The database uses HNSW indexing on vector columns to optimize semantic searches.
- Tuning parameters organize vectors into graphs, avoiding slow sequential table scans.
- The database queries note chunks and retrieves matching paragraphs in milliseconds.
- [SCREENSHOT: SQL script executing EXPLAIN ANALYZE on a similarity vector query, showing index use]

```
-- HNSW index DDL schema
CREATE INDEX idx_note_chunks_embedding
  ON note_chunks
  USING hnsw (embedding vector_cosine_ops)
  WITH (
    m = 16,
    ef_construction = 64
  );
```

Scaling and Database Backup Design

- The database architecture separates static university catalogs from user notes.
- Database partitions structure planner task tables, keeping query performance stable.
- Database backups ensure data recovery in case of system failures.

- [SCREENSHOT: Supabase DB settings panel showing backup configurations and status trackers]

```
-- Daily logs database view DDL
CREATE VIEW user_daily_summary AS
  SELECT
    user_id,
    COUNT(id) FILTER (WHERE is_completed) as done,
    COUNT(id) as total
  FROM daily_tasks
  GROUP BY user_id;
```